couplr: Optimal Matching with Verifiable Certificates and Column Generation

by Gilles Colling

Abstract Matching two groups of units so that the total cost of the chosen pairs is as small as possible is the linear assignment problem, which sits underneath causal-inference matching and experimental design. Two things bound what software built on it can offer. An answer arrives without a proof: a solver reports the state it terminated in, not a checkable statement about the matching. And the graph of admissible pairs is built before it is solved, so its memory sets the size ceiling. We present the couplr package, which addresses both. Every flow-representable matching design compiles into one internal flow model, so node potentials come back from all of them and can be checked against the linear-programming optimality conditions. On those potentials the package optimises over a restricted graph: each unit starts with its nearest admissible partners, the omitted pairs are priced against the potentials the restricted solve returns, and pairs enter until none prices in, at which point those potentials bound how far the result can sit from the complete problem's optimum. Matching 16,667 treated units to 33,333 controls holds 0.29 percent of the pairs and returns an optimum certified within that bound. Warm starting the same loop traces a matching path over a swept constraint at roughly half the cost of solving each point independently. A data-frame interface, nineteen assignment algorithms, balance diagnostics and cardinality matching share the same core. couplr is available on CRAN.

1 Introduction

Many data-analysis tasks come down to the same question: given two sets of objects and a cost attached to every possible pair, which pairing has the smallest total cost? A researcher building an observational study pairs treated units with controls that resemble them on measured covariates, a scheduler assigns staff to shifts, an ecologist pairs surveyed plots across two time points, and an image-processing pipeline maps the pixels of one picture onto the pixels of another. In each case the mathematical object is the linear assignment problem, and the quality of the answer depends on solving it to optimality rather than greedily.

R has good software at both ends of this problem, in two separate groups of packages. On the statistical side, **MatchIt** (Ho et al., 2011) provides a common interface to several matching designs and delegates optimal pair and full matching to **optmatch** (Hansen, 2007), which formulates those designs as minimum-cost flow problems (Hansen and Klopfer, 2006), reaches RELAX-IV or one of four LEMON algorithms through its `solver` argument, and stores sparse distances as an `InfinitySparseMatrix` whose non-finite entries are the pairings it will not consider. **Matching** (Sekhon, 2011) provides multivariate and propensity-score matching, with `GenMatch()` using genetic search to optimise covariate weights. **designmatch** (Zubizarreta et al., 2024) formulates balance-constrained designs as integer programs. **rcbalance** (Pimentel, 2022) matches under refined covariate balance constraints (Pimentel et al., 2015), taking a distance structure that lists the permissible controls of each treated unit and solving the resulting network as a minimum-cost flow problem. **quickmatch** (Sävje et al., 2024) aims at large data sets with generalized full matching, which its own description calls near-optimal. **cobalt** (Greifer, 2025) supplies balance diagnostics across packages. What these expose is the matching design; the assignment problem underneath is reached through a solver argument at most, and alternative optima and a bottleneck objective sit outside all of them. Statements about other packages here and in Table 6 were assessed on 2026-08-09 against **MatchIt** 4.7.2, **optmatch** 0.10.8, **Matching** 4.10-15 and **designmatch** 0.5.5, and against the reference manuals of **rcbalance** 1.8.8 and **quickmatch** 0.2.3 on 2026-08-26.

On the algorithmic side, **clue** (Hornik, 2024) exposes the Hungarian method through `solve_LSAP()`, and **lpSolve** (Berkelaar et al., 2024) solves the assignment problem as a general linear program. These return an optimum for a cost matrix the user has already built; covariate handling, calipers and balance tables sit outside their scope.

A workflow that needs both halves therefore threads several packages by hand. Matching on observed covariates instead of a propensity score, imposing calipers on more than one variable at a time, blocking, and choosing the solver deliberately are all available in R, each in a different package.

Two further limits are common to both groups, and they are the ones this article is about. The first concerns what a result says about itself. A solve returns a matching and a status, and the status records the state the solver terminated in. Whether that matching is the optimal one is a separate question with a checkable answer, since linear programming supplies optimality conditions a third party can

test on the returned solution. The general linear programming interfaces return material for that test in their own terms: **lpSolve** returns a `duals` component from `lp()` and `lp.assign()` when `compute.sens` is set, indexed by the rows and columns of the program as assembled, so reading those numbers as assignment potentials and checking the conditions against a cost matrix is left to the caller. Among the matching-specific interfaces surveyed, none exposes assignment or flow potentials together with a function that verifies optimality against them. **optmatch** states bounds in their place, an `exceedances` attribute and a solver tolerance, both described with the capability table below.

The second concerns size. Sparse matching is a developed literature, and what its methods have in common is that the sparse network is settled before the matching is solved over it. **rcbalance** takes a per-unit list of permissible controls built from exact groups and a caliper. Yu et al. (2020) choose a propensity-score caliper by an iterative form of Glover's algorithm and match optimally in the much sparser graph that leaves, and Yu and Rosenbaum (2022) grade the potential pairings and admit lower grades only as far as they are needed. In each case the rule deciding which pairs exist is fixed in advance of the solve that uses them, and a network too sparse drops pairings it should have considered. What is missing is a rule read off the solution in hand, and a statement about the pairs it never built.

We present the **couplr** package (Colling, 2026), which treats the assignment problem itself as the organising object and builds the workflow around it. A user who starts from a data frame with covariates and a treatment indicator, and a user who starts from a cost matrix, enter through different functions and reach the same solver core. We contribute five things to R:

1. A common compilation layer. One-to-one, k-to-one, with replacement, blocked and full matching are the same internal flow model under different arc capacities, so what is written once serves all of them.
2. Certificates rather than reported status. `verify_assignment()` and `verify_flow()` check primal feasibility, dual feasibility, complementary slackness and objective equality on a solution, reporting each separately and independently of the solver that produced it, in exact arithmetic where the potentials allow and naming the arithmetic the verdict was reached in.
3. Dual-certified adaptive edge generation. Under `memory_mode = "implicit"` the complete graph is never built: a restricted problem is solved, the omitted pairs are priced against its duals, violating pairs are added, and the loop repeats until none prices in, at which point the duals bound the result's distance from the complete problem's optimum, and certify optimality where that bound closes.
4. Warm-started matching paths. `match_path()` sweeps one constraint over a sequence of values, resuming each point from the one before it, with a certificate per point.
5. Nineteen assignment algorithms written from scratch in C++ and reachable by name, covering augmenting-path, auction, cost-scaling and network-flow families, with k-best and bottleneck assignment, entropy-regularised transport, and cardinality matching that reports an optimality gap against the largest sample its constraints admit.

The article is arranged for two kinds of reader. The assignment engine and the matching interface built on it come first and stand on their own; the optimisation architecture, its implementation and the measurements follow for a reader interested in where the certificates and the column generation come from.

Results were produced with couplr version 1.7.0.

2 The core assignment engine

The assignment problem

Let $C \in \mathbb{R}^{n \times m}$ be a cost matrix whose entry C_{ij} is the cost of pairing row $i \in \{1, \dots, n\}$ with column $j \in \{1, \dots, m\}$, and take $n \leq m$; couplr transposes the problem internally when it is not and maps the assignment back afterwards, so the user never sees the transpose. Write $X \in \{0,1\}^{n \times m}$ for the assignment matrix, with $X_{ij} = 1$ when row i is matched to column j. The linear assignment problem is

$$\min_X \sum_{i=1}^{n} \sum_{j=1}^{m} C_{ij} X_{ij} \quad \text{subject to} \quad \sum_j X_{ij} = 1 \ \forall i, \qquad \sum_i X_{ij} \leq 1 \ \forall j. \tag{1}$$

Every row is matched and every column is used at most once. The row constraint is an equality: relaxing it to $\sum_j X_{ij} \leq 1$ would leave $X = 0$ optimal for any non-negative cost matrix, which is not the problem a matching application poses.

Forbidden pairs are encoded as $C_{ij} = \infty$, and enough of them make (1) infeasible, since no

assignment can then reach every row. couplr solves a lexicographic relaxation in that case: the largest number of matched rows first, then the smallest total cost among the assignments attaining that number. The section on constraints below describes how that problem is reached and what it costs.

The constraint matrix of (1) is totally unimodular, so the linear programming relaxation has integral optima (Birkhoff, 1946; Burkard and Çela, 1999) and the combinatorial problem can be solved by primal-dual methods rather than by branch and bound. The dual assigns a potential u_i to each row and v_j to each column, and is

$$\max_{u,v} \sum_{i=1}^{n} u_i + \sum_{j=1}^{m} v_j \quad \text{subject to} \quad u_i + v_j \le C_{ij} \; \forall (i,j), \qquad v_j \le 0 \; \forall j. \tag{2}$$

The sign condition is what the column inequality in (1) costs: the dual objective sums v over every column while an assignment pays only for the ones it uses. When $n = m$ none can be left out, so the column inequalities may be replaced by equalities without changing the feasible set, and the equality formulation leaves v unrestricted in sign; `verify_assignment()` imposes $v_j \le 0$ only when $n < m$, since Jonker-Volgenant returns free-sign potentials on a square problem and they certify optimality there.

Complementary slackness ties a pair of potentials to a particular X:

$$X_{ij} = 1 \implies u_i + v_j = C_{ij}, \qquad \textstyle\sum_i X_{ij} = 0 \implies v_j = 0. \tag{3}$$

The reduced cost $C_{ij} - u_i - v_j$ is a lower bound on what forcing pair (i, j) would add to the total rather than the whole of it, because forcing one pair can displace others.

Verifiable optimality certificates

Dual feasibility on its own says nothing about a particular X. What certifies optimality is X primal feasible, (u, v) feasible for (2), and both halves of (3) holding; in exact arithmetic those three settle the objectives as well, so objective equality is a reported check rather than a condition of its own. `verify_assignment()` reports the four separately, so a failure names the check it failed on.

```
set.seed(2)
cost <- matrix(runif(120), nrow = 6, ncol = 20)
verify_assignment(assignment(cost), cost)
```

```
#> Assignment certificate
#> ======================
#>
#>   primal_feasible          TRUE
#>     valid matching         TRUE
#>     all rows matched       TRUE
#>   dual_feasible            TRUE
#>   complementary_slackness  TRUE
#>     matched arcs tight     TRUE    (max slack 0.000e+00)
#>     unmatched columns free TRUE    (max |v_j| 0.000e+00)
#>   duality_gap              0.000000e+00
#>   max_suboptimality        0.000000e+00
#>   certified_optimal        TRUE
#>   arithmetic               exact, no tolerance
#>
#>   primal objective  0.2821665586
#>   dual objective    0.2821665586
#>   matched           6 of 6 rows, 20 columns
```

Two properties make this a check rather than a report. It is written against the solution rather than the solver, taking an assignment, a cost matrix and a pair of potentials and never consulting the code that produced them. And optimal duals are shared by all optimal solutions of a linear program, so a matching from one solver can be certified against potentials from another, which is what puts the solvers returning no duals of their own inside its scope.

Complementary slackness is where a plausible verifier goes wrong, and the rectangular case is where it shows. Asserting the first half of (3) alone, tightness on matched pairs, accepts solutions that are not optimal: in our own prototyping a perfect, dual-feasible, matched-tight solution passed such a

check with its primal cost 0.4% above the optimum, because one unmatched column retained $v_j < 0$ and contributed a spurious term to the dual objective. `verify_assignment()` asserts both halves and reports them as `cs_matched_tight` and `cs_unmatched_free`.

The arithmetic the conditions are decided in is the other place where a certificate can overstate what it knows. Each of them is the sign of $C_{ij} - u_i - v_j$, and every quantity in that expression is a double, which is a rational number, so each sign has an exact answer. Deciding them within a band around zero instead makes the verdict a statement about a neighbourhood of the instance rather than about the instance, and what the band costs can be written down. A certificate at tolerance ϵ accepts potentials that violate dual feasibility by ϵ on any pair, miss tightness by ϵ on any matched arc, and leave $|v_j| \le \epsilon$ on any unused column. Shifting the potentials down by ϵ restores exact dual feasibility and the sign condition, and bounds the matching's excess over the optimum by $(n + 2m)\epsilon$; the supplementary material carries the derivation. At the largest shape in the benchmark below and the default $\epsilon = 10^{-9}$, that is 8×10^{-5} in the cost units.

Setting the band to zero is not the fix, because the sign of the double evaluation can itself be wrong: a reduced cost far smaller than the potentials it is formed from can round to zero, and in the negative case that is a dual infeasibility a zero-band check would accept. `verify_assignment()` evaluates the sign exactly instead: it splits each rounded addition into the sum and the part rounding lost, giving an expansion of doubles equal to the difference in the reals, and takes the sign of that expansion (Shewchuk, 1997). A rounding-error bound decides which pairs the double evaluation already settles, so the expansion runs near tightness rather than on all nm.

Removing the band from the two conditions removes it from the conclusion. Objective equality is then not a fourth measurement but a consequence: with matched arcs tight and unmatched columns free, the primal cost is the sum of u over matched rows plus the sum of v over used columns, which is the dual objective exactly when every row is matched. The software reports the gap it computes as an independent numerical cross-check on that reasoning.

An exact certificate states that the matching attains the minimum total cost of the matrix as supplied, and is available when the potentials are exactly optimal for it, which is a property of the arithmetic that produced them. Over ten instances per cell, integer costs and uniform costs on the unit interval gave one in every instance at 50, 200 and 800 rows square, at 200 rows against 600 columns, and for each of the eleven solvers we ran at 200 by 200. Euclidean distances between normal clouds gave one in none: those potentials miss exact tightness by a relative 2×10^{-16} to 5×10^{-15}, which is what rounding in the arithmetic that produced them costs, and there the certificate decides the same conditions within a stated tolerance and says so. Every instance was certified in one arithmetic or the other, and a verdict names which; `arithmetic = "exact"` refuses the fallback for a caller who wants the stronger statement or nothing.

The nineteen-solver portfolio

Every solver here targets the objective of (1), so the choice among them is a performance question rather than a statistical one, and it depends on the shape of the problem. The methods that reach it by scaling real costs onto an integer grid are exact for the instance that scaling produces, which is the supplied one where the costs are integers and a rounding of it otherwise; `verify_assignment()` is what decides whether a returned matching is optimal for the matrix as supplied. We group them into four families and a set of special-purpose routines.

Augmenting-path methods grow a matching one row at a time, each time finding a shortest augmenting path in the residual graph under reduced costs. The Hungarian method (Kuhn, 1955; Munkres, 1957) is the classical member; the Jonker-Volgenant algorithm (Jonker and Volgenant, 1987) adds column reduction and auction-style initialisation, which is what makes it fast in practice on dense problems despite an $O(n^3)$ worst case. `lapmod` is the sparse variant, working from an adjacency list so that forbidden pairs cost nothing to carry.

Auction methods (Bertsekas, 1988) let rows bid for columns, raising prices until no row wants to switch. They converge to optimality once the bid increment ε falls below $1/n$ for integer costs, and ε-scaling gives the useful practical variants. Auction is naturally parallel and tolerant of near-degenerate cost structures where augmenting-path methods spend time on long paths.

Cost-scaling methods solve a sequence of increasingly precise problems, each time halving the scale on which costs are rounded and reusing the previous solution (Goldberg and Kennedy, 1995). The Gabow-Tarjan bit-scaling algorithm (Gabow and Tarjan, 1989) belongs here. On a graph with $|V|$ nodes and $|E|$ edges it reaches $O(\sqrt{|V|}\,|E| \log(|V|C_{\max}))$ for integer costs, where $C_{\max} = \max_{i,j} |C_{ij}|$. A complete bipartite graph on n and m units has $|V| = n + m$ and $|E| = nm$, so the bound there is $O(\sqrt{n+m}\; nm \log((n+m)C_{\max}))$. We are not aware of a prior publicly available open-source implementation of this 1989 bipartite assignment algorithm.

Bit-scaling runs on integers, so the conversion is part of the method and its rule is stated rather than left to the caller. Finite costs are shifted so the smallest is zero, multiplied by a scale factor and rounded, with the factor set to keep the scaled costs clear of the sentinel marking a forbidden pair and the path sums inside 64-bit arithmetic; a matrix whose range leaves no usable factor is refused with the limit named. Integer costs convert exactly and the optimum returned is the optimum of the instance as supplied. Costs that are not integers are solved on the rounded instance, and the supplementary material states the scale factor, the rounding rule and the bound on how far the returned matching can sit above the optimum of the matrix as supplied. Potentials are divided by the factor and shifted back, so they belong to the original matrix, and whether the matching is optimal for it rather than only for the rounded instance is a question `verify_assignment()` answers.

Network-flow methods treat (1) as a minimum-cost flow problem on a bipartite network (Ahuja et al., 1993) and apply push-relabel (Goldberg and Tarjan, 1988), cycle cancelling, network simplex, or Orlin's strongly polynomial algorithm (Orlin, 1993). These are the most general formulations, which makes them the right base for variable-ratio designs where a group absorbs several units. On a plain one-to-one problem they are the slowest choice.

Special-purpose routines cover cases where the general machinery is wasted: exhaustive enumeration below nine units, Hopcroft-Karp bipartite matching (Hopcroft and Karp, 1973) when costs are binary or constant, Ramshaw-Tarjan (Ramshaw and Tarjan, 2012) for strongly rectangular problems where padding to a square matrix would dominate the work, and bottleneck assignment, which minimises the largest pair cost instead of the sum.

The binary case is worth stating in full, since a maximum-cardinality routine carries no costs and the problem does. When every admissible edge holds the same value, all perfect matchings cost the same and any maximum-cardinality matching is optimal. When the finite costs are $\{0, 1\}$, the routine runs Hopcroft-Karp on the zero-cost edges alone: a perfect matching there totals zero, which no assignment improves on. If that subgraph admits no perfect matching the optimum needs at least one unit edge, and the problem passes to the weighted solver on the original matrix. The specialised path answers only where its answer is the optimum.

Two extensions return something other than a single optimal assignment. Murty's ranking procedure (Murty, 1968) with Lawler's partition scheme (Lawler, 1972) enumerates the k cheapest assignments, which lets a user see how much of the total cost is forced and how much is arbitrary. Sinkhorn iteration (Cuturi, 2013) solves the entropy-regularised relaxation, returning a doubly stochastic coupling rather than a permutation.

Matrix-level interface and extensions

The solver layer is reachable on its own, for a user who arrives with a cost matrix. `lap_solve()` and `assignment()` take a matrix and return the optimal assignment, and `lap_solve()` also accepts a long data frame of source, target and cost columns, the natural shape when costs come from a database. The `method` argument names any of the nineteen solvers and defaults to `"auto"`; it accepts twenty names, since `"ssp"` is a second spelling of `"sap"`.

```
set.seed(1)
cost <- matrix(sample.int(100, 25, replace = TRUE), nrow = 5)
res  <- lap_solve(cost)
res
```

```
#> Assignment Result
#> =================
#>
#> # A tibble: 5 x 3
#>   source target  cost
#>    <int>  <int> <int>
#> 1      1      4     7
#> 2      2      2    14
#> 3      3      1     1
#> 4      4      3    54
#> 5      5      5    44
#>
#> Total cost: 120
#> Method: bruteforce
```

Dual potentials come from `assignment_duals()`, which returns u and v alongside the assignment, so passing that result to `verify_assignment()` certifies the matching without a second solve. Four

extensions operate on the same matrix: `lap_solve_kbest()` returns the k cheapest assignments in increasing order of cost, `bottleneck_assignment()` minimises the largest pair cost rather than the sum, `lap_solve_batch()` solves a stack of independent problems, optionally across threads, and `sinkhorn()` returns the entropy-regularised coupling, which `sinkhorn_to_assignment()` rounds to a permutation.

```
kb <- lap_solve_kbest(cost, k = 3)
tapply(kb$total_cost, kb$rank, unique)

#>   1   2   3
#> 120 120 150
```

The two cheapest assignments both cost 120 and the third costs 150, so the optimum here is attained by two distinct pairings. A matching study reading that output learns its matched sample is not unique, which stays invisible when a solver returns one assignment. The solver portfolio and the dispatcher that selects within it are measured across a grid of cost regimes, admissibility patterns and aspect ratios in the empirical evaluation.

3 Matching with couplr

A matching workflow

The package ships `hospital_staff`, a synthetic personnel dataset supporting a treated/control workflow without external data: 200 units against 300, with age, experience, hourly rate, department, certification level and a full-time indicator.

`match_couples()` is the front door. It takes the two data frames and the covariates to match on and returns the optimal one-to-one pairing under the requested distance; `auto_scale = TRUE` screens the covariates for degenerate variance and excessive missingness and picks a scaling method, so that variables measured in years and in ordinal levels contribute comparably.

```
library(couplr)
data(hospital_staff)
treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)
covars  <- c("age", "experience_years", "certification_level")

m <- match_couples(treated, control, vars = covars, auto_scale = TRUE)
m$pairs[1:3, c("left_id", "right_id", "distance", ".age_diff")]

#> # A tibble: 3 x 4
#>   left_id right_id distance .age_diff
#>   <chr>   <chr>       <dbl>     <dbl>
#> 1 1       260         0.217        -1
#> 2 2       294         0.175         2
#> 3 3       212         0             0
```

The call matches all 200 treated units at a total distance of 59.95. Beside the identifiers and each pair's distance, `m$pairs` carries a signed per-variable difference for every matching variable, so which covariate drives a poor pair is visible without recomputing anything; `m$unmatched` holds what is left over on each side and `m$info` records the method, the pair count, the total distance and the estimand.

`balance_diagnostics()` computes standardised mean differences, variance ratios and Kolmogorov-Smirnov statistics on the matched sample, and `balance_table()` formats them: experience and certification fall below the conventional 0.1 threshold on the absolute standardised difference (Stuart, 2010) and age sits slightly above at 0.116. `join_matched()` returns the analysis-ready frame, one row per pair, with every column of both inputs under `_left` and `_right` suffixes.

The matched frame is the input to an outcome analysis rather than the end of one. A paired difference taken on it estimates an adjusted association, and reading that as a causal effect is what rests on assumptions no matching method can verify: conditional ignorability, positivity, and the stable unit treatment value assumption. What matching can offer against the first is `sensitivity_analysis()`, which implements Rosenbaum bounds for the Wilcoxon signed-rank statistic on one-to-one matched pairs (Rosenbaum, 2002), reporting p-value bounds across a grid of the odds-ratio parameter Γ so that a reader can judge how strong an unmeasured confounder would have to be to overturn the result. The supplementary material carries both on the matched sample above.

Calipers, forbidden pairs and partial matching

Three mechanisms restrict which pairs are admissible, and all three work by writing non-finite entries into the cost matrix before it reaches the solver. A global `max_distance` forbids any pair beyond a distance ceiling. Per-variable `calipers` forbid pairs whose raw difference on a named variable exceeds a threshold, independently of the distance metric in use. Explicit forbidden pairs are passed as non-finite entries by the user. Every solver recognises a non-finite entry as a missing edge, so no returned pair ever violates a constraint.

Tight constraints usually make (1) infeasible. couplr first drops the rows and columns whose every edge is forbidden, reporting them unmatched. The remaining submatrix can still fail Hall's condition, with several rows competing for the same few columns and no assignment reaching every row. Rather than return whichever pairs a heuristic happens to find, couplr replaces the forbidden entries with a finite sentinel σ and re-solves with the same optimal solver.

Write $[\ell, h]$ for the range of the admissible costs of the pruned submatrix and k for the smaller of its two dimensions.

Lemma 1. *If $\sigma > k(|\ell| + |h|)$, then a padded assignment using more sentinel edges than another costs strictly more.* The proof is in the supplementary material.

A minimum-cost solve of the padded problem therefore uses as few sentinel edges as any assignment can, which is a maximum-cardinality admissible matching, and among the assignments attaining that number it minimises the real cost. Dropping the sentinel pairs leaves the lexicographic optimum: the largest admissible matching, cheapest among those of its size. couplr takes $\sigma = (k+1)(|\ell| + |h|) + 1$, which satisfies the hypothesis in every sign case, and solves a maximisation by negation with the sentinel negated alongside it. No sentinel enters a reported cost.

Above a padded objective of 2^{53} couplr refuses the padding rather than returning an ordering its arithmetic may not support: `match_couples()` warns and falls back to a greedy partial matching, saying that it is not optimal, and `assignment(cardinality = "maximum")` errors and asks for rescaled costs or an explicit `unmatched_penalty`. That threshold is a precondition on the solve and not a proof of it. What establishes that a particular padded solve reached its optimum is the certificate, and the lemma carries that verdict across to the lexicographic objective; the supplementary material sets out both.

```
m_cal <- match_couples(treated, control, vars = covars, auto_scale = TRUE,
                       calipers = list(age = 3, experience_years = 2),
                       max_distance = 1.5)
```

The caliper costs pairs: 197 of the 200 treated units keep a partner. That is a caliper working, and the reason the unmatched identifiers are returned rather than dropped, since the units that fail to match define the population the estimate applies to. The constraints here are severe, leaving 2,173 admissible pairs out of 60,000, so the padded solve is what produces this result; a greedy pass over the same edges matches 180 units rather than 197.

Blocking is the other structural constraint. `block_id` restricts matching to levels of a factor, and `matchmaker()` builds blocks when no grouping variable exists, from an existing variable or by k-means or hierarchical clustering on chosen block variables. The solver runs once per block, and blocked designs report per-block balance so imbalance inside one stratum is not averaged away across the others.

Matching designs

Beyond one-to-one matching, `match_couples()` takes `ratio` for k-to-one designs and `replace` for matching with replacement. Five further designs have their own entry points, each returning an object in the same family. `ps_match()` estimates a propensity score (Rosenbaum and Rubin, 1983) and matches on it, with the caliper expressed in standard deviations of the linear predictor. Where the constraints admit it, `full_match()` assigns every unit to a matched group of variable size, so no unit is discarded. A group holds either one left unit and several right ones or one right unit and several left ones, and both shapes appear in the same solution, which is full matching in the sense of Hansen and Klopfer (2006). The default solves the corresponding minimum-cost flow problem with Johnson potentials (Johnson, 1977); a two-pass greedy heuristic is reached by asking for it, under `method = "greedy"`. `cem_match()` implements coarsened exact matching (Iacus et al., 2012), and `subclass_match()` divides the sample into propensity-score subclasses, returning subclass weights for the average treatment effect on the treated or on the whole sample.

`cardinality_match()` targets the cardinality-matching objective (Zubizarreta et al., 2014), the largest matched sample meeting prespecified balance constraints, with total distance ordered after

cardinality; which solver answers it follows from the kind of balance asked for. Fine and refined covariate balance (Rosenbaum et al., 2007; Pimentel et al., 2015), stated through `fine` and `refined`, are representable in the network as category nodes, so such a call reaches a single minimum-cost flow solve returning the largest balanced sample with a dual certificate at polynomial cost. Linear moment constraints, a finite `max_std_diff` or an explicit `moments` argument, are not, and are dualised and searched by branch and bound under `node_limit` and `time_limit`. Either way the result reports `n_matched` beside `best_possible`, the `gap` between them, whether the answer is `certified`, and what the search `stopped_on`. `best_possible` is the search's current upper bound on attainable sample size, and is the largest sample the constraints admit once `certified` is `TRUE`. A pruning loop is available as `engine = "heuristic"` and reports its gap as unknown, which is the honest reading of what a heuristic establishes.

Interoperability

couplr fits into a workflow built around the established packages. `as_matchit()` converts a couplr result into a `matchit` object and `match_data()` returns data in the layout **MatchIt** users expect, so downstream code keeps working; `bal.tab()` methods are registered for the couplr classes, so **cobalt** produces its usual balance output; and weighted matched data flows into **marginaleffects** (Arel-Bundock, 2025).

4 Common optimisation architecture

Three of the package's capabilities are consequences of one decision. Compiling every flow-representable design into a single flow model is what makes node potentials available from all of them; the potentials are what certify a solution; a certificate is what makes it safe to optimise over a restricted graph of pairs; and a certified restricted solve is what a warm start resumes from:

$$\text{common flow duals} \rightarrow \text{certificates} \rightarrow \text{safe edge generation} \rightarrow \text{warm-started paths.}$$

Flow compilation

Underneath the designs of the previous section there is one object. It has nodes, each carrying a signed supply, and arcs, each carrying a cost and a capacity range. As Table 1 sets out, a one-to-one match, a k-to-one match, matching with replacement and a full match differ in the bounds their arcs receive, not in the code that solves them. Stating full matching as a minimum-cost flow problem is established (Hansen and Klopfer, 2006); here it is the one design whose flow value is not fixed in advance, since how many pair arcs a full matching uses depends on how many groups it forms, so the source injects the unit count and a bypass arc absorbs the rest. What the flow model contributes is that every design in the package that a flow can represent compiles into it, so one solver and one certificate serve all of them, with the warm-start machinery shared where the public interface exposes it. Those are the six rows of the table: one-to-one, k-to-one, with replacement, blocked and exact, full matching, and fine and refined balance. The designs that sit outside it sit outside because their answer is not a flow: `cem_match()` coarsens and strata are read off directly, `subclass_match()` cuts the propensity score into subclasses, and the moment constraints of `cardinality_match()` are dualised and searched by branch and bound rather than represented in a network.

Table 1: How each matching design is stated in the internal flow model. The designs differ in the capacity bounds their arcs receive and in how many networks are built, rather than in which solver runs.

Design	Flow formulation
One-to-one matching	Unit-capacity bipartite flow
k-to-one matching	Capacitated bipartite flow, treated supply k
Matching with replacement	Control capacities above one
Blocked and exact matching	Independent networks per stratum
Full matching	Edge cover, both sides bounded below by one
Fine and refined balance	Category nodes with capacity bounds

Compiling a design, solving it, and checking the answer stay three separate steps, because the third is worth something only while it is separate: a solver status is not itself an independently checkable

certificate. The flow and the potentials go into `verify_flow()` as inputs to a check, and the arcs the flow is checked against are stated independently of it.

```
prob <- list(n_nodes = 4, supply = c(2, 1, -2, -1),
             arcs = data.frame(tail = c(1, 1, 2, 2), head = c(3, 4, 3, 4),
                               lower = 0, upper = 2, cost = c(1, 3, 2, 1)))
cert <- verify_flow(c(2, 0, 0, 1), prob, potential = c(0, 0, 1, 1))
c(certified = cert$certified_optimal, gap = cert$duality_gap)

#> certified       gap
#>         1         0
```

Two solvers read that network. One is successive shortest paths with Johnson potentials (Johnson, 1977), reachable as `method = "csflow"`. The other, `method = "push_relabel"`, is cost-scaling push-relabel in the successive approximation of Goldberg and Tarjan (1990): a sequence of ε-optimal flows, each phase dividing ε, saturating the arcs the smaller ε no longer admits, and clearing the excess with two local operations. A push moves excess along an arc of negative reduced cost; a relabel lowers the price of a node holding excess and having no such arc, by exactly the amount that gives it one. Below $\varepsilon = 1/(n+1)$ an ε-optimal flow on integer costs is optimal, which ends the scaling, so real costs are scaled to integers first.

Tolerance is where a flow certificate can overstate what it knows, and the scale of the potentials sets it. A design that ranks objectives lexicographically by weighting them, as `cardinality_match()` ranks cardinality above balance above distance, reaches potentials in the millions, where one unit in the last place is around 10^{-9} and an exactly optimal flow cannot meet an absolute tolerance of that size. The comparisons therefore scale with the arc's own magnitude, and the widest tolerance any comparison used comes back as `dual_tolerance`, so the verdict names the resolution it was reached at.

Node potentials therefore come back from every design rather than from the one-to-one case alone. Those potentials are what the next section prices against.

Certified implicit edge generation

A matching problem on n treated units and m controls has nm admissible pairs, and materialising them is what sets the size ceiling: at 50,000 against 100,000 the dense cost matrix alone is 40 GB. The lazy representation removes the matrix while leaving the search over all m columns intact. What `memory_mode = "implicit"` changes is the problem the solver ranges over: it optimises over a restricted pair set and certifies the result against the complete one, so the pairs outside that set are priced rather than searched.

The loop is column generation (Lübbecke and Desrosiers, 2005) over the pair variables of (1). A restricted master problem holds a subset of them, its optimal duals price the rest at $C_{ij} - u_i - v_j$, and the restricted solution solves the full problem as soon as no omitted variable prices negative. One thing separates it from the textbook setting: the omitted variables are enumerated rather than returned by an optimisation oracle, because a pair variable is an index rather than a combinatorial object to be searched for. Write $E_t \subseteq \{1,\dots,n\} \times \{1,\dots,m\}$ for the candidate set at round t and ϵ for the pricing threshold.

1. Seed E_0 with each row's w nearest admissible columns, w read off m as $6\lceil \log_2 m \rceil$ and capped at m.
2. Solve (1) restricted to E_t with the flow model of the previous section, warm-started from the incumbent flow and prices when $t > 0$. If it comes back short of a complete matching, repair the shortfall as below and repeat this step.
3. Read the master's potentials as assignment duals (u, v).
4. Price the omitted pairs: $\bar{C}_{ij} = C_{ij} - u_i - v_j$ for admissible $(i, j) \notin E_t$.
5. Stop if $\bar{C}_{ij} \geq -\epsilon$ everywhere.
6. Otherwise set $E_{t+1} = E_t \cup V_t$, where V_t holds each row's most negative violators, and return to step 2.

The stopping rule is the certificate.

Proposition 1. *Let X solve (1) restricted to a pair set E with optimal duals (u, v), and suppose $C_{ij} - u_i - v_j \geq 0$ for every admissible $(i, j) \notin E$. Then X is optimal for the complete problem.*

Proof. Optimality on E gives complementary slackness for X and (u, v) there, so every matched pair is tight and every unmatched column has $v_j = 0$, and the cost of X is $\sum_i u_i + \sum_j v_j$. The hypothesis

extends dual feasibility from E to every admissible pair, so (u, v) is feasible for (2) on the complete problem and that sum is a lower bound on the cost of every assignment of it. X is one of them and attains the bound. □

Nothing in the argument is numerical: the hypothesis is the sign of $\bar{C}_{ij}$ on the omitted pairs, and the conclusion is equality, not proximity. In practice four quantities weaken it, and the supplementary material accounts for them separately: the gap δ the master's own certificate reports, the pricing threshold ϵ, the sign condition on the column potentials, and the allowance the pricing bound carries. Their sum bounds the matching's excess over the complete optimum. At $\epsilon = 10^{-9}$ and the largest shape in the benchmark below, the threshold's contribution $n\epsilon$ is 2×10^{-5} in the cost units, and every run reported there came back with $\delta = 0$. The certificate returns that sum as `max_suboptimality`, beside `certified_reduced_cost_floor`, the floor the pricing scan proves for every admissible pair, which sits below the minimum it observed exactly where pruning supplied part of the proof.

The loop terminates because each round adds at least one pair E_t did not hold, so $|E_t|$ strictly increases under the bound nm. That bound is the worst case: duals that price most of the grid in make the loop build the complete graph and pay for the rounds that got there. `max_rounds`, 60 by default, is a guard rather than the termination argument, and a run reaching it returns `iteration_limit` rather than a certificate.

A pricing sweep stores nothing: the reduced cost of a pair is consumed on the spot, so the memory the mode holds is the candidate set, nw pairs from the seed plus what the rounds add. How it reads the pairs depends on the cost source. Where the metric admits a ball bound the columns are held in a metric tree, and a subtree whose bound cannot beat the row's current threshold is discarded without being visited; a source with no such bound is read one pair at a time, at $O(nm)$ arithmetic to the sweep. Mahalanobis distance is Euclidean after whitening and always pays for the bound, being quadratic in the covariates, while the metrics linear in them take a tree only in low dimension. Where the bound prunes, the prune certifies that its subtree holds no violating edge, and the prunes with the leaves examined certify the pricing result. The supplementary material states the bound, the summaries a node carries and the allowance every bound is lowered by. Pruning does not put a round below one pass over the pairs, since the seed evaluates what it seeds with and a pair priced in one round is priced again in the next, so `edges_evaluated` can exceed `possible_edges` where the tree is working; what the loop saves is the graph.

Rows contribute at most `keep_per_row` violators each, five by default, so a round adds at most $5n$ pairs and never fewer than one. Selection is by strict improvement on a row's worst kept reduced cost over a scan running columns ascending, so ties keep the lower column index and the same input gives the same candidate set at every round.

A master that comes back short of a complete matching has no dual solution to price with, so the shortfall is repaired rather than priced. Hall's condition (Hall, 1935) names a row set S whose neighbourhood $N(S)$ in the restricted graph is too small to go round. Every restricted edge out of S already lands in $N(S)$, so only columns outside it can move the deficiency, and the re-seed reads the full source over the rows of S and takes columns from outside $N(S)$. When there are none, $N(S)$ is that neighbourhood in the complete problem too, and the result is infeasible with $(S, N(S))$ as its witness, naming the units short of partners rather than only the fact.

So the restricted solution is optimal for a problem that was never built, and `certify = TRUE` runs that check and returns it.

Two quantities in `$search` describe what that cost. `candidate_edges` against `possible_edges` is the graph built against the complete one that was not, and `edges_evaluated` counts distance evaluations over all rounds.

Round count is the quantity to control, since each round prices every omitted pair again, and it is what the seed rule of step 1 is sized for: a seed too narrow buys what it needs a round at a time, a seed too wide pays for columns the matching never uses. The rule clears the two-round floor, one seed and one sweep that finds nothing to add, on the problems below, and a run reports the width it used as `$search$seed_width`.

```
m_imp <- match_couples(treated, control, vars = covars, auto_scale = TRUE,
                       memory_mode = "implicit", certify = TRUE)
m_imp$certificate$certified_optimal
```

```
#> [1] TRUE
```

```
unlist(m_imp$search[c("seed_width", "n_rounds",
                      "candidate_edges", "possible_edges")])
```

```
#>      seed_width        n_rounds candidate_edges  possible_edges
#>              54               2           10800           60000
```

The certificate the loop returns is the numerical one, which is why the ϵ version of Proposition 1 is the statement it carries. Dual feasibility over the pairs the master omits comes from the pricing bounds, which ask whether anything prices below $-\epsilon$ rather than evaluating each omitted pair, so the exact conclusion of the certification section is not reachable from a scan assembled that way, and the certificate reports that as its arithmetic.

Constraints tighten the loop rather than complicating it, since a caliper or a distance ceiling is checked before a distance is computed and removes pairs from the set to be priced.

Two neighbours are worth separating from this. Abeywickrama et al. (2021) also decline to build a complete cost matrix, deferring exact edge costs behind an inexpensive, application-specific lower bound, so what they save is arithmetic. The bound here is geometric rather than application-specific, and what the loop saves first is the graph it never builds; where the metric admits a ball bound the pricing also declines to evaluate a pair, which is where the two meet. Graded matching (Yu and Rosenbaum, 2022) also enlarges a sparse network only as far as it has to, but a grade is fixed before the match, while the reduced cost that admits a pair here is a property of the solution the master has just returned, so the loop stops on a statement about the complete graph rather than on the grades running out.

The mode is available on `match_couples()` and `assignment()`, and it is requested rather than selected: `memory_mode = "auto"` does not choose it, because the worst case above is a real one. Its scope is the unit-capacity assignment. A full match's column nodes carry capacities above one, so a column dual there is not the assignment dual this pricer reads. A general flow arc does carry a reduced cost of its own, from the potentials at the nodes it joins, so pricing one is available in principle; it is this implementation that stops at the assignment. How small a graph the loop ends up holding, how many pricing rounds it takes and how it behaves on geometries built to be hostile to it are measured in the empirical evaluation.

Warm-started matching paths

A caliper is rarely known in advance. The usual practice is to sweep it, watch what the matched sample and its balance do, and choose a point, which means solving the same problem many times over. `match_path()` solves the sweep as one sequence.

```
p <- match_path(treated, control, vars = covars, auto_scale = TRUE,
                vary = "max_distance", values = c(1.0, 1.2, 1.5, 2.0, 3.0))
p$path[, c("max_distance", "n_matched", "total_distance", "certified")]
```

```
#> # A tibble: 5 x 4
#>   max_distance n_matched total_distance certified
#>          <dbl>     <int>          <dbl> <lgl>
#> 1          1         200           62.6 TRUE
#> 2          1.2       200           61.5 TRUE
#> 3          1.5       200           59.9 TRUE
#> 4          2         200           59.9 TRUE
#> 5          3         200           59.9 TRUE
```

The sweep shows where the constraint stops binding: the total distance falls until 1.5 and does not move above it, since a cut that wide forbids nothing the unconstrained optimum uses. Reading that off a path is the point of solving the sweep rather than one value.

Each point resumes from the matching the point before it found. Raising the distance cut admits pairs while leaving that matching feasible, so what remains is to repair reduced-cost violations on the arcs that just became admissible, which is one round of the pricing loop of the previous section, where an independent call at that value needs a full solve. One mechanism serves the sweep and the single solve. A value whose admissible pairs cannot support a complete matching is reported rather than approximated, coming back with no matching and the Hall witness beside it, so a sweep that starts below the feasibility threshold says where that threshold is. The ascending direction is a correctness requirement rather than a convention: a descending sweep withdraws pairs the current matching may be standing on, which breaks feasibility and needs a repair phase the loop does not have, so a descending sequence is refused with a message saying why.

The return value is a `couplr_path`, whose `$path` carries a row per point with the matched count, total distance, seconds, rounds and whether it was certified, and whose `$balance` carries a row per point per variable. `certify = TRUE` is the default here, and a point's certificate is what says its matching is the optimal one at that value rather than the one the warm start happened to reach. What a sweep costs against solving each point independently is measured in the empirical evaluation.

Memory representations

The dense representation of the cost matrix is the binding constraint on problem size long before runtime is. A 100,000 by 100,000 problem needs 80 GB as doubles; the same units described by 100 covariates need 80 MB. couplr exposes this as `memory_mode`, with values `"dense"`, `"lazy"`, `"implicit"` and `"auto"`, differing in what the solver ranges over. The dense path holds the complete matrix; the lazy path holds none and still considers every pair, evaluating each distance when it is asked for, over Jonker-Volgenant and auction with the built-in metrics; the implicit path runs the edge-generation loop described above. All three solve the same problem, and on the instances reported below they returned the same pairing wherever the pairing was compared.

Under `"auto"` the package decides. Below ten million cells, about 80 MB dense, it skips the check, since probing free memory on every ordinary problem would be pure overhead. Above that it estimates what a dense solve peaks at rather than what the matrix occupies, at 10 times the raw cell bytes, and compares that against available system memory read per platform. The empirical evaluation reports the peaks that multiplier sits above. If the estimate exceeds half of what is available, `"auto"` switches to the lazy path where one exists and warns where one does not, naming blocking, greedy matching or a smaller problem. Detection failure is treated as unknown rather than as success: a fixed 4 GB threshold takes over, so the guard never silently disappears on an unrecognised platform.

5 Implementation and verification

The package is organised in three layers. The first turns user input into a cost source: it validates the data frames, screens covariates, applies scaling and weights, and settles how distances are obtained and which pairs are admissible. The second compiles the requested design into the flow model and solves it, which for the plain one-to-one case is (1). The third builds result objects and their methods. The object model, the C++ organisation of the nineteen solvers and the dependency footprint are in the supplementary material.

Automatic solver selection

`method = "auto"` inspects the problem and picks a solver, and the rules are few.

1. Problems with at most eight rows and eight columns go to exhaustive enumeration, which is exact and faster than setting up a general solver.
2. Cost matrices whose finite entries are constant or lie in $\{0,1\}$ go to the Hopcroft-Karp style routine, which exploits the absence of a real cost scale.
3. Everything else goes to Jonker-Volgenant.

Sparsity and aspect ratio carried rules of their own, sending a mostly forbidden matrix to `lapmod` and a wide one to shortest augmenting paths. Both were slower than the default in the regime they were written for, so both were removed; the regime grid behind Table 4 is the measurement, and both solvers stay reachable by name.

Rule 2 reads the entries, and so does the rejection of `NaN` inputs that runs for every method, so choosing a solver costs one C++ pass over the cost matrix, returning every quantity the rules need without allocating and without a second scan. That leaves the inspection a linear read in front of a superlinear solve, so its share of the runtime falls as problems grow, and the dashed line in Figure 1 sits on the solver the dispatcher selects. The pass is skippable for callers who already know the shape of their problem, by naming the method.

Independent verification

Certificates are the check that survives leaving the test suite. `verify_assignment()` and `verify_flow()` run in $O(nm)$ and $O(\text{arcs})$ respectively without solving anything, so a user can run them on their own problem, in exact arithmetic or at their own tolerance, against a solution the package produced.

Testing and reproducibility

A smoke test establishes that a function returns an object of the right shape; for a solver the standard is whether the assignment is the optimal one. Every optimal solver is checked against exhaustive enumeration on randomised instances covering integer and fractional costs, square and rectangular shapes, minimisation and maximisation, and forbidden edges. The reference is the same `bruteforce`

solver the dispatcher uses below nine units, so the verified and the shipped path are one implementation. Constrained matching is checked against an enumeration over all partial matchings of a small instance, which supplies both halves of its lexicographic objective. The paths that avoid materialising the problem are checked against the path that materialises it: the implicit loop and the dense solve must return the same total wherever the dense solve runs at all, and a warm-started point must return what an independent solve at that value returns. Pair identity is the stronger check and is reported where it holds rather than required, since an instance with several optima admits more than one answer of equal cost; what correctness asks for is a certified objective the two agree on. The next section reports both comparisons on the sizes it covers.

The suite covers the R layer, the solver bodies compiled outside R, the C++ wrappers, dispatch rules and the matching designs.

Every number the next section reports comes from a script that ships with the article. Each caches its CSV under `paper/` and re-measures only when that file is moved aside; `paper/run_bench_suite.sh` runs them in order on one machine, which is how these numbers were produced, and the scaling, implicit, implicit-grid, regime and memory scripts take `--quick` for a reduced grid. The environment block beside the CSVs records the machine, the R version, the package version and the commit.

6 Empirical evaluation

Edge generation, robustness and memory

Table 2 reports the three memory modes on those same problems. The dense and lazy arms are pinned to Jonker-Volgenant, so they differ only in representation; the implicit arm carries the flow model as its restricted master by construction, which is a difference between the arms and is the definition of the mode rather than a choice.

Table 2: One-to-one optimal Mahalanobis matching by memory mode, wall-clock seconds on a single core of an Apple M4 Pro. Graph built is the arcs the implicit loop ended up holding as a share of the complete problem's. Distances is its evaluations over all rounds as a multiple of the complete pair count, so a pair priced twice counts twice. All three arms were timed in one session; the dense arm was run at the four sizes where its pairing can be compared against the loop's, and the scaling table below carries dense timings at the two largest sizes. Every cell returned the same total distance.

Problem size	dense	lazy	implicit	Graph built	Distances	Rounds
167 + 333	21 ms	8 ms	11 ms	16.22%	2.00	2
667 + 1,333	161 ms	53 ms	59 ms	4.95%	2.00	2
1,667 + 3,333	901 ms	350 ms	310 ms	2.16%	1.96	2
3,333 + 6,667	3.17 s	1.59 s	1.15 s	1.17%	1.88	2
6,667 + 13,333	not run	6.38 s	4.1 s	0.63%	1.71	2
16,667 + 33,333	not run	61.9 s	21.3 s	0.29%	1.42	2

Three things in that table go together. The loop holds a small graph, 16.22% of the pairs at the smallest size and 0.29% at the largest, a share that falls as the problem grows since the pairs a matching can use grow more slowly than the pairs a problem has. It pays for that in arithmetic, at 1.42 to 2.00 times the complete pair count in distance evaluations, so at eight covariates it computes more distances than one full sweep would. And it is still the fastest arm at four of the six sizes, because what it avoids is not the arithmetic but the graph. At 16,667 + 33,333 it finishes in 21 seconds against 62 for the lazy path. At the two smallest sizes the lazy path is faster; at 667 + 1,333 it takes 53 ms against 59 ms, which is where the loop's fixed costs are still visible against a problem this size.

Every run in the table settled in 2 rounds, one seed and one sweep that found nothing to add and certified. Every one came back certified with a duality gap of zero. On the four sizes where the dense solve can also be run, the two agree to within 3.4e-13, and the pairing is identical unit by unit, so the loop returns the assignment rather than an assignment of the same cost.

The implicit column is a representation and a solver at once, so comparing it with the dense column reads both differences together. At 3,333 + 6,667 the materialised matrix takes 3.17 seconds under Jonker-Volgenant and 4.15 under the flow solver the restricted master belongs to, against 1.15 for the loop, so what the loop saves is not what a change of solver would give. The supplementary material carries the decomposition.

One cloud is not a characterisation, and the table above reports one. The supplementary material sweeps the loop around that cloud one factor at a time: covariate counts from 2 to 32, seven point

clouds including clustered points, a displaced treated group, coordinates on a coarse lattice so that distances tie, one contested core where every treated unit's nearest controls are the same few, and a shell where every treated-to-control distance is nearly equal, three caliper settings including the tightest value each problem is still feasible under, and five sizes, each at several seeds. Across the 20 cells that grid holds, the loop settled in 2 to 10 rounds and ended holding between 0.29 and 73.32 percent of the complete pair count, at 0.24 to 6.67 times that count in distance evaluations. All 20 cells certified, at a largest absolute duality gap of 3.64e-12, and 18 of the 18 cells small enough to run the dense path beside the loop returned the same pairing unit by unit. The margin over the lazy path is where the clouds separate: it runs from 10.83x on shifted to 1.04x on heavy_tailed.

Two rounds is what an ordinary cloud costs and it is not what the loop is bounded by, so the grid's hostile cell is worth reading on its own. In the contested core, where every treated unit's nearest controls are the same few, the duals travel a long way before they stop pricing pairs in. At 10,000 units it takes 9 to 10 rounds, ends holding 73 percent of the complete pair count rather than a fraction of one percent, and evaluates 6.7 times that count in distances. It still certifies, and it is still 3.2 times faster than the lazy path on this cloud. What it shows is the worst case of the previous section happening: the graph the loop ends up holding is most of the one it set out to avoid, and it has paid for the rounds that built it. This is why the mode is requested rather than selected.

The claim that memory rather than time is what bounds a dense matching has so far been made with edge shares, which are a proxy for it. Peak resident set size is the quantity itself, and it is a property of a process rather than of an expression, so each arm here is a fresh R session that loads what it needs, runs one matching and exits, measured from outside. A session that loaded the same packages and matched nothing is the floor each arm is read against.

At 6,667 + 13,333 the dense path peaks 6256 MB above that floor, the lazy path 22 MB and the loop 128 MB, against 27429 MB for **optmatch** and 8641 MB for **MatchIt**. Two readings follow. The loop is not the smallest of the three: the lazy path holds 22 MB against the loop's 128 MB, since it stores no graph at all while the loop stores the candidate set it has built. What the loop buys over it here is time, and the ceiling both of them remove is the dense path's. The second reading is the guard's. Building the matrix and stopping there accounts for 1093 MB on a matrix whose entries occupy 711 MB, so holding it costs 1.5 times its cells and solving over it costs 8.8 times them. That gap is preparation copies and the solver's own workspace, and it is what the automatic mode has to read: a guard sized on the matrix would let a dense solve start on a machine it does not fit. `estimate_dense_matrix_mb()` estimates the matrix at 4 times the raw cell bytes and `estimate_dense_solve_mb()` the peak of the solve at 10 times them, and `memory_mode = "auto"` compares the second against available memory. Neither multiplier is fitted to one cell. Across 5,000, 10,000 and 20,000 units the solve peaked at 10.5, 7.2 and 8.8 times the cell bytes and the matrix at 1.8, 1.7 and 1.5 times them. Both ratios are read from 5,000 units up: at 2,000 the matrix holds 7 MB of cells against a session floor near 146 MB, so what the peak measures there is the session rather than the solve, and both multipliers are exceeded on it. The matrix default bounds every one of the sizes it is read against. The largest solve peak, at 5,000 units, reached 10.5 times the cells against the 10 the guard assumes, so at that size the estimate came in 20 MB under the peak it is meant to bound. The guard exists to refuse a solve that will not fit rather than to predict where a peak lands, so the multiplier has to sit above the ratios it is read against, and at this size it does not. Every arm and size is in the supplementary material.

Warm-started paths

Table 3 reports a sweep of 20 caliper values solved as one path against the same 20 values solved independently. The grid is the data's own: its tight end is the tightest caliper the problem still admits a complete matching under, found by bisection, and its wide end is the median treated-to-control distance, which admits half the pairs and constrains nothing.

The table leads with the wall clock for the whole sweep, which is the number a user meets: it carries the feasibility check, the balance reading each point takes, the conversions in and out of the solver and the rest of the R-level work. Both sides pay that per point, since an independent solve here is a path of one value rather than a separate entry point, so the comparison is like for like. The solver's own time sits beside it as a diagnostic, saying how much of the saving is the warm start rather than the sweep's shared setup.

The saving comes from the rounds column. A warm-started point starts from a matching that is still feasible at the new value, so what it has to repair are the reduced-cost violations on the arcs the wider cut just admitted, which the loop reaches in about one round per point. A point solved independently starts from a seed and pays for the seeding sweep as well.

Correctness is the part of this comparison that has to hold before the timing means anything. Every point on the path returns the status and the matched count its independent solve returns, and the totals agree to 0 relative, so the path reproduces the feasibility statuses, matched counts and objective

Table 3: A caliper sweep solved as one warm-started path against the same values solved independently, end-to-end wall clock for the whole sweep on a single core of an Apple M4 Pro. Wall ratio is the independent sweep's time over the warm-started path's, so a value above one is the path running faster. Solve-time ratio is the same comparison on the seconds the solver reports for itself, summed over the points, which excludes the R-level work both sides do. Rounds is the pricing rounds each side took, summed over the sweep.

Problem size	As a path	Independently	Wall ratio	Solve-time ratio	Rounds
167 + 333	0.059 s	0.139 s	2.36x	2.75x	22 / 40
667 + 1,333	0.534 s	1.09 s	2.05x	2.06x	23 / 40
1,667 + 3,333	3.1 s	6.44 s	2.08x	2.08x	21 / 40
6,667 + 13,333	44.1 s	89.1 s	2.02x	2.02x	21 / 40

values of independent solves. Pair identities were not compared, and where a value admits more than one optimum equal totals do not make the matchings equal.

Solver portfolio and automatic dispatch

Figure 1 reports median wall-clock solve time against problem size for all nineteen solvers, grouped by family. We draw costs as integers uniformly from $[1, 10{,}000]$ on square matrices and report medians of five replicates on a single core of an Apple M4 Pro. The slower families are capped at smaller sizes so that the scalable solvers can be measured to $n = 5000$ within a reasonable budget. All timings in this section were measured on 2026-09-01 under R 4.5.3 with couplr 1.7.0 at commit 3a083b1, **optmatch** 0.10.8 and **MatchIt** 4.7.2.

Jonker-Volgenant with its warm start is the fastest general-purpose solver at every measured size, which is why it is the dispatcher's default. The auction and cost-scaling families are competitive at small sizes and fall behind on dense uniform costs, where their advantage, tolerance of degenerate structure and a better asymptotic bound for Gabow-Tarjan, does not pay. General flow formulations are the slowest, as expected of machinery solving a larger problem than the one posed. The two special-purpose routines run on the inputs they exist for, exhaustive enumeration below nine units and the Hopcroft-Karp style solver on binary costs, so their timings are not comparable with the rest and are shown only to establish that each is practical on its own input. These orderings hold for dense

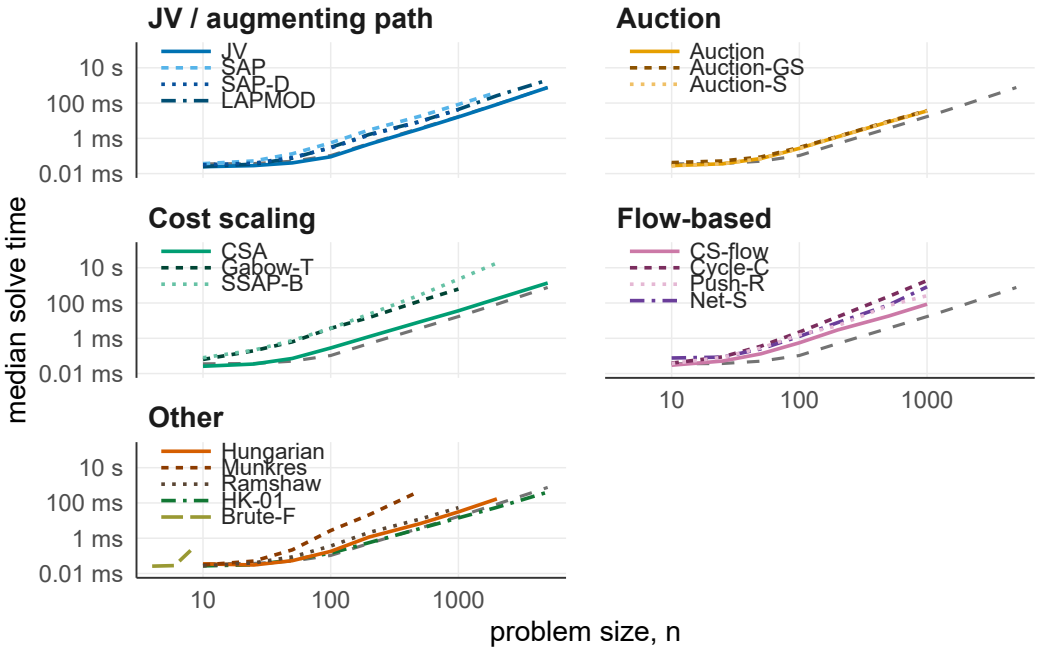

Figure 1: Median wall-clock solve time against problem size for the nineteen assignment solvers in couplr, grouped by algorithm family on shared log-log axes. The two special-purpose solvers in the Other panel run on their own inputs and are not comparable to the rest: HK-01 is timed on binary cost matrices, and Brute-F only up to n = 8. The dashed grey line repeated in every panel is the automatic dispatcher on the uniform integer costs.

uniform integer costs on square matrices, which is the whole of what the figure establishes, and they fix the dispatcher's default. Each label in the figure is a solver's `method` name: JV `jv`, SAP `sap`, SAP-D `sap_dense`, LAPMOD `lapmod`, Auction `auction`, Auction-GS `auction_gs`, Auction-S `auction_scaled`, CSA `csa`, CS-flow `csflow`, Push-R `push_relabel`, Cycle-C `cycle_cancel`, Net-S `network_simplex`, SSAP-B `ssap_bucket`, Gabow-T `gabow_tarjan`, Ramshaw `ramshaw_tarjan`, Hungarian `hungarian`, Munkres `munkres`, HK-01 `hk01` and Brute-F `bruteforce`.

The grid behind Table 4 is what reduced the rules to three. It crosses the properties the solver figure holds fixed: whether the finite costs have a scale worth exploiting, how much of the matrix is forbidden and whether what remains is one component, and how far from square the problem is. Eight cost regimes, from uniform integers through binary, constant, heavily tied and heavy-tailed costs to Euclidean distances over uniform and clustered points, are crossed with six admissibility patterns, complete, unstructured at four densities from 60 percent of the entries finite down to 1 percent, and four disconnected components, and with three aspect ratios from square to one row per ten columns. That crossing is the base tier. The large tier repeats a subset of it at three times the size rather than the whole: the regimes and patterns where a rule that holds at 500 rows might fail at 1500, against the solvers that reach that size inside the budget. The table therefore covers 189 cells, 144 of them the complete base crossing and 45 the large-tier subset. Each cell is generated as several independent instances, each instance is timed several times, and every solver in the panel sees the same instance. The solver `"auto"` selects is timed as a solver of its own, so its number carries the probing pass the rules cost, and it is read against the fastest named solver in the same cell.

Table 4: Each dispatch rule over the cells of the regime grid where it fired. Solver is what the rule selects. Fastest counts the cells where that solver was also the quickest of the panel. Median and worst are the time the dispatched solver took over the time the cell's fastest named solver took, so 1.00x is a rule that picked the best available solver. The large tier times a reduced panel that omits the special-purpose solvers, so a cell whose dispatched solver is not in it reads below 1.00x.

Rule	Solver	Cells	Fastest	Median	Worst
no earlier rule applies	jv	144	59	1.14x	10.41x
no cost scale to exploit	hk01	45	34	1.00x	2.18x

Over the 189 cells the dispatcher picked the fastest solver of the panel in 93 of them, at a median 1.07x of the cell's best time. A cell reads below `1.00x` for one of two reasons. The large tier times a reduced panel, so on its 9 binary cells the solver the dispatcher named was not among those it is read against; on the 36 cells of that rule whose panel does hold its solver the median is 1.01x. Elsewhere the two sides are separate timings of one solver and scatter by a few percent either way. The two rules that were removed can be read off the same grid. Over the 96 cells with at least three columns per row, where the rectangular rule used to fire, the default takes a median 1.04x; over the 96 cells with more than half the entries forbidden, 1.23x. Timed against the solver each rule named rather than against the panel's best, `sap` takes a median 1.41x of the default's time on the wide cells and `lapmod` 1.09x of it on the sparse ones, so in each case the rule was naming the slower of the two.

Where the default is behind, it is behind in one cost regime. Over the 18 cells whose finite costs are heavily tied it takes a median 3.92x of the fastest solver in the cell and as much as 10.41x; over the remaining 126 cells of the rule the median is 1.06x and the worst 3.90x. The solvers that win the tied cells are `network_simplex` in 9 of them and `csflow` in 5, both of them the flow formulations that Figure 1 puts last on dense uniform costs. No rule in the table reads how tied the costs are and none was added for it: fitting one to this grid would score it on the measurements that chose it.

The full grid, every solver in every cell, is in the supplementary material.

Comparison with established alternatives

Balance

Speed only matters once the packages agree on the answer. We fix the task in Figure 2, one-to-one optimal matching on a Mahalanobis distance with pooled within-group covariance, and run it on the LaLonde NSW data (LaLonde, 1986; Dehejia and Wahba, 1999), 185 treated units, 429 controls and eight covariates. The pooled within-group convention is the one `optmatch::match_on()` uses by default and the one couplr adopts.

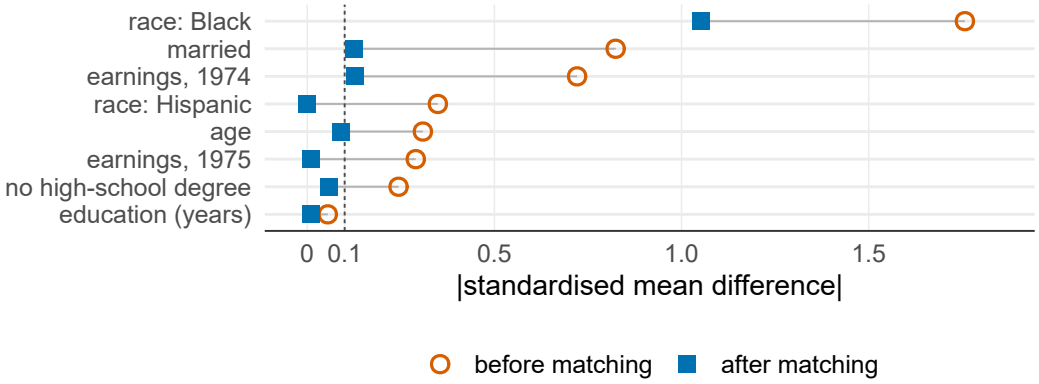

Figure 2: Absolute standardised mean differences on the eight LaLonde NSW covariates, before and after one-to-one optimal Mahalanobis matching with pooled within-group covariance. couplr, MatchIt and optmatch agree to three decimal places after matching, so a single matched point is shown per covariate. The dashed line marks the conventional 0.1 threshold. The indicator for Black respondents starts far outside the plotted range at 1.757 and remains at 1.053 after matching, a residual imbalance that no one-to-one matcher can resolve on this data.

The three packages produce identical absolute standardised mean differences to three decimal places, so one matched point is plotted. Marginals could still hide different assignments, so we score all three on one objective: the same Mahalanobis distance matrix, summed over each package's chosen pairs. Each matches all 185 treated units, at a total distance of 304.042 for couplr against 304.043 for both alternatives, a relative difference of 3×10^{-6}. That is agreement on the objective to the precision reported here rather than a demonstration that all three attain the same minimum, and separating those two readings is what a certificate is for: couplr's matching comes back certified optimal for this cost matrix, and the same check runs on either alternative's pairing against the same duals.

Five of the eight covariates fall below 0.1 after matching, `married` and 1974 earnings sit just above at 0.124 and 0.128, and the indicator for Black respondents drops from 1.757 to 1.053 without approaching it. That is a property of this dataset rather than of any package: the control pool does not hold enough Black respondents for a one-to-one match to balance that indicator.

Scaling

Table 5 reports wall-clock time on synthetic problems with the same eight-covariate structure, at six sizes with a one-to-two ratio of treated to control units, so every problem here is rectangular. Every package in this comparison materialises the distance matrix, so we time couplr with `memory_mode = "dense"`. Each size is generated as several independent instances with the timing repetitions inside them, and the table reports the median and the interquartile range across instances: repetitions of one matrix measure the clock, and what varies between problems is what a reader wants to know.

Table 5: One-to-one optimal Mahalanobis matching, wall-clock time by problem size. Treated to control ratio 1:2, eight covariates, pooled within-group covariance, single core with single-threaded BLAS on an Apple M4 Pro. Each cell is the median over independently generated instances of that size, with the interquartile range across instances in brackets and the timing repetitions taken inside each instance. The 50,000 row carries one instance and is a single run rather than a median, so no range is reported for it. The optmatch option max.problem.size was set to Inf for the two largest sizes. int overflow marks an integer-size overflow inside the optmatch back end reached through MatchIt; timeout marks a run exceeding the 600-second cap.

Problem size	couplr	optmatch	MatchIt
167 + 333	20 ms (20-21)	116 ms (116-121)	138 ms (136-139)
667 + 1,333	152 ms (141-164)	1.75 s (1.67-1.76)	2.27 s (2.24-2.29)
1,667 + 3,333	788 ms (778-814)	12.8 s (12.7-12.8)	16.2 s (16.2-16.2)
3,333 + 6,667	3.15 s (3.1-3.22)	60.5 s (59.8-60.6)	75.3 s (74.6-75.5)
6,667 + 13,333	11.6 s (11.1-12)	286 s (283-290)	int overflow
16,667 + 33,333	75.7 s	timeout	timeout

The margin widens with problem size. couplr is about 6 times faster than **optmatch** at $n = 500$

and 25 times faster at $n = 20{,}000$, where it finishes in 11.6 seconds against 4.8 minutes. **MatchIt** runs 7 to 24 times slower up to $n = 10{,}000$, which is the largest size it completes before `matchit(method = "optimal")` aborts inside the **optmatch** back end with an integer-size overflow. At $n = 50{,}000$ couplr finishes in 76 seconds and both alternatives exceed the 600-second cap.

What the table compares is three workflows on one task, from data frame to matched pairs, not three implementations of one algorithm. The packages do not state the same problem: **optmatch** solves a minimum-cost flow problem, which the full and variable-ratio designs it is built around need and a plain one-to-one problem does not, and **MatchIt** reaches that same back end, while couplr's dispatcher sends this problem to Jonker-Volgenant. Reading the margin as a faster inner loop would need a benchmark holding the formulation fixed across packages, which this one does not.

The lazy path removes the memory ceiling as well. The same $n = 50{,}000$ match under `memory_mode = "lazy"` completes in 62 seconds without ever building the distance matrix, returning the assignment the dense path returns. At $n = 20{,}000$ it takes 6.4 seconds against 11.6 for the dense path, so on these problems recomputing distances is cheaper than materialising and traversing the matrix.

Feature coverage

Table 6 lists where the three packages differ. Rows where all three offer the feature, covariate distance, propensity-score matching, exact blocking and a cost-matrix entry point, are omitted.

Table 6: Selected assignment-layer features, omitting areas where the alternatives lead, among them the breadth of designs reachable through a single MatchIt call and the maturity of optmatch's full matching. All three accept a user-supplied cost matrix; the third row records what a pair match reaches the solver as, couplr the rectangular assignment directly and the other two through fullmatch(). The supplementary material gives what each entry was read from, and the versions assessed.

Feature	couplr	MatchIt	optmatch
Per-variable calipers from a named vector	yes	yes	partial
User-supplied cost matrix accepted	yes	yes	yes
Pair matching solved as an assignment	yes	via full match	via full match
Sparse cost support	yes	yes	yes
k-best assignments	yes	no	no
Bottleneck (minimax) assignment	yes	no	no
Rosenbaum sensitivity bounds	yes	no	no
Public dual-potential verifier	yes	no	gap bound
User-selectable assignment algorithm	19 solvers	via optmatch	5 algorithms

7 Summary and discussion

Making the assignment problem the organising object, rather than a hidden step inside a matching function, is what lets one package serve a causal-inference user starting from covariates and an operations-research user starting from a cost matrix. Every flow-representable design compiles into one flow model, so node potentials come back from all of them, those potentials turn a reported status into a certificate, and the certificate is what makes it safe to solve a problem that was never built.

What that architecture buys is measurable: at 16,667 + 33,333 the restricted graph settled at 0.29% of the complete pair count and returned a certified optimum, and over the 189 cells of the regime grid the dispatcher ran at a median 1.07x of the cell's fastest solver.

couplr has five limitations:

- The edge-generation loop's scope is the unit-capacity assignment: a full match's column nodes carry capacities above one, so its column duals are not the duals the pricer reads and `full_match()` has no implicit path. That is a limit of this implementation rather than of the method, since a general flow arc has a reduced cost from the potentials at its ends.
- The lazy cost path covers one-to-one matching without replacement under Jonker-Volgenant and auction with the built-in metrics. Replacement, a ratio above one, `cardinality_match()` and a user-supplied distance function build the dense matrix.
- `cardinality_match()` answers a moment-constrained problem by branch and bound under node and time limits, so a run can stop with a gap rather than a certificate and reports which it reached. Fine and refined balance is answered certified.

- The dispatcher reads two properties and otherwise defaults to Jonker-Volgenant, at a median 1.06x of the cell's fastest solver over most of the regime grid. Where the finite costs are heavily tied it takes a median 3.92x and as much as 10.41x, behind `network_simplex` and `csflow`. Fitting a rule to this grid would score it on the measurements that chose it, so the default stands.
- Matching on observed covariates cannot address unmeasured confounding, which the sensitivity analysis quantifies rather than removes.

couplr is available on CRAN and developed at `https://github.com/gcol33/couplr`, with documentation at `https://gillescolling.com/couplr/`. Bug reports and feature requests go through the GitHub issue tracker. No external funding supported this work.

Bibliography

T. Abeywickrama, V. Liang, and K.-L. Tan. Optimizing bipartite matching in real-world applications by incremental cost computation. *Proceedings of the VLDB Endowment*, 14(7):1150–1158, 2021. doi: 10.14778/3450980.3450983. [p11]

R. K. Ahuja, T. L. Magnanti, and J. B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993. [p5]

V. Arel-Bundock. *marginaleffects: Predictions, Comparisons, Slopes, Marginal Means, and Hypothesis Tests*, 2025. URL https://CRAN.R-project.org/package=marginaleffects. R package version 0.29.0. [p8]

M. Berkelaar et al. *lpSolve: Interface to Lp_solve v. 5.5 to Solve Linear/Integer Programs*, 2024. URL https://CRAN.R-project.org/package=lpSolve. R package version 5.6.23. [p1]

D. P. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14:105–123, 1988. doi: 10.1007/BF02186476. [p4]

G. Birkhoff. Tres observaciones sobre el algebra lineal. *Universidad Nacional de Tucumán Revista Series A*, 5:147–151, 1946. [p3]

R. E. Burkard and E. Çela. Linear assignment problems and extensions. *Handbook of Combinatorial Optimization*, pages 75–149, 1999. doi: 10.1007/978-1-4757-3023-4_2. [p3]

G. Colling. *couplr: Optimal Pairing and Matching via Linear Assignment*, 2026. URL https://CRAN.R-project.org/package=couplr. R package version 1.7.0. [p2]

M. Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL https://papers.nips.cc/paper_files/paper/2013/hash/af21d0c97db2e27e13572cbf59eb343d-Abstract.html. [p5]

R. H. Dehejia and S. Wahba. Causal effects in nonexperimental studies: Reevaluating the evaluation of training programs. *Journal of the American Statistical Association*, 94(448):1053–1062, 1999. doi: 10.1080/01621459.1999.10473858. [p16]

H. N. Gabow and R. E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989. doi: 10.1137/0218069. [p4]

A. V. Goldberg and R. Kennedy. An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, 71(2):153–177, 1995. doi: 10.1007/BF01585996. [p4]

A. V. Goldberg and R. E. Tarjan. A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4):921–940, 1988. doi: 10.1145/48014.61051. [p5]

A. V. Goldberg and R. E. Tarjan. Finding minimum-cost circulations by successive approximation. *Mathematics of Operations Research*, 15(3):430–466, 1990. doi: 10.1287/moor.15.3.430. [p9]

N. Greifer. *cobalt: Covariate Balance Tables and Plots*, 2025. URL https://CRAN.R-project.org/package=cobalt. R package version 4.6.3. [p1]

P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, s1-10(1):26–30, 1935. doi: 10.1112/jlms/s1-10.37.26. [p10]

B. B. Hansen. Optmatch: Flexible, optimal matching for observational studies. *R News*, 7(2):18–24, 2007. URL https://journal.r-project.org/articles/RN-2007-014/. [p1]

B. B. Hansen and S. O. Klopfer. Optimal full matching and related designs via network flows. *Journal of Computational and Graphical Statistics*, 15(3):609–627, 2006. doi: 10.1198/106186006X137047. [p1, 7, 8]

D. E. Ho, K. Imai, G. King, and E. A. Stuart. MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, 42(8):1–28, 2011. doi: 10.18637/jss.v042.i08. [p1]

J. E. Hopcroft and R. M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973. doi: 10.1137/0202019. [p5]

K. Hornik. *clue: Cluster Ensembles*, 2024. URL https://CRAN.R-project.org/package=clue. R package version 0.3-66. [p1]

S. M. Iacus, G. King, and G. Porro. Causal inference without balance checking: Coarsened exact matching. *Political Analysis*, 20(1):1–24, 2012. doi: 10.1093/pan/mpr013. [p7]

D. B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1): 1–13, 1977. doi: 10.1145/321992.321993. [p7, 9]

R. Jonker and T. Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38(4):325–340, 1987. doi: 10.1007/BF02278710. [p4]

H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2 (1-2):83–97, 1955. doi: 10.1002/nav.3800020109. [p4]

R. J. LaLonde. Evaluating the econometric evaluations of training programs with experimental data. *The American Economic Review*, 76(4):604–620, 1986. URL https://www.jstor.org/stable/1806062. [p16]

E. L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. *Management Science*, 18(7):401–405, 1972. doi: 10.1287/mnsc.18.7.401. [p5]

M. E. Lübbecke and J. Desrosiers. Selected topics in column generation. *Operations Research*, 53(6): 1007–1023, 2005. doi: 10.1287/opre.1050.0234. [p9]

J. Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957. doi: 10.1137/0105003. [p4]

K. G. Murty. An algorithm for ranking all the assignments in order of increasing cost. *Operations Research*, 16(3):682–687, 1968. doi: 10.1287/opre.16.3.682. [p5]

J. B. Orlin. A faster strongly polynomial minimum cost flow algorithm. *Operations Research*, 41(2): 338–350, 1993. doi: 10.1287/opre.41.2.338. [p5]

S. D. Pimentel. *rcbalance: Large, Sparse Optimal Matching with Refined Covariate Balance*, 2022. URL https://CRAN.R-project.org/package=rcbalance. R package version 1.8.8. [p1]

S. D. Pimentel, R. R. Kelz, J. H. Silber, and P. R. Rosenbaum. Large, sparse optimal matching with refined covariate balance in an observational study of the health outcomes produced by new surgeons. *Journal of the American Statistical Association*, 110(510):515–527, 2015. doi: 10.1080/01621459.2014.997879. [p1, 8]

L. Ramshaw and R. E. Tarjan. On minimum-cost assignments in unbalanced bipartite graphs. Technical Report HPL-2012-40R1, HP Laboratories, 2012. URL https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html. [p5]

P. R. Rosenbaum. *Observational Studies*. Springer Series in Statistics. Springer, 2 edition, 2002. doi: 10.1007/978-1-4757-3692-2. [p6]

P. R. Rosenbaum and D. B. Rubin. The central role of the propensity score in observational studies for causal effects. *Biometrika*, 70(1):41–55, 1983. doi: 10.1093/biomet/70.1.41. [p7]

P. R. Rosenbaum, R. N. Ross, and J. H. Silber. Minimum distance matched sampling with fine balance in an observational study of treatment for ovarian cancer. *Journal of the American Statistical Association*, 102(477):75–83, 2007. doi: 10.1198/016214506000001059. [p8]

F. Sävje, J. S. Sekhon, and M. Higgins. *quickmatch: Quick Generalized Full Matching*, 2024. URL https://CRAN.R-project.org/package=quickmatch. R package version 0.2.3. [p1]

J. S. Sekhon. Multivariate and propensity score matching software with automated balance optimization: The Matching package for R. *Journal of Statistical Software*, 42(7):1–52, 2011. doi: 10.18637/jss.v042.i07. [p1]

J. R. Shewchuk. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363, 1997. doi: 10.1007/PL00009321. [p4]

E. A. Stuart. Matching methods for causal inference: A review and a look forward. *Statistical Science*, 25(1):1–21, 2010. doi: 10.1214/09-STS313. [p6]

R. Yu and P. R. Rosenbaum. Graded matching for large observational studies. *Journal of Computational and Graphical Statistics*, 31(4):1406–1415, 2022. doi: 10.1080/10618600.2022.2058001. [p2, 11]

R. Yu, J. H. Silber, and P. R. Rosenbaum. Matching methods for observational studies derived from large administrative databases. *Statistical Science*, 35(3):338–355, 2020. doi: 10.1214/19-STS699. [p2]

J. R. Zubizarreta, R. D. Paredes, and P. R. Rosenbaum. Matching for balance, pairing for heterogeneity in an observational study of the effectiveness of for-profit and not-for-profit high schools in Chile. *The Annals of Applied Statistics*, 8(1):204–231, 2014. doi: 10.1214/13-AOAS713. [p7]

J. R. Zubizarreta, C. Kilcioglu, and J. P. Vielma. *designmatch: Matched Samples that are Balanced and Representative by Design*, 2024. URL https://CRAN.R-project.org/package=designmatch. R package version 0.5.5. [p1]

Gilles Colling
University of Vienna
Department of Botany and Biodiversity Research
Rennweg 14, 1030 Vienna, Austria
https://gillescolling.com
ORCID: 0000-0003-3070-6066
gilles.colling051@gmail.com